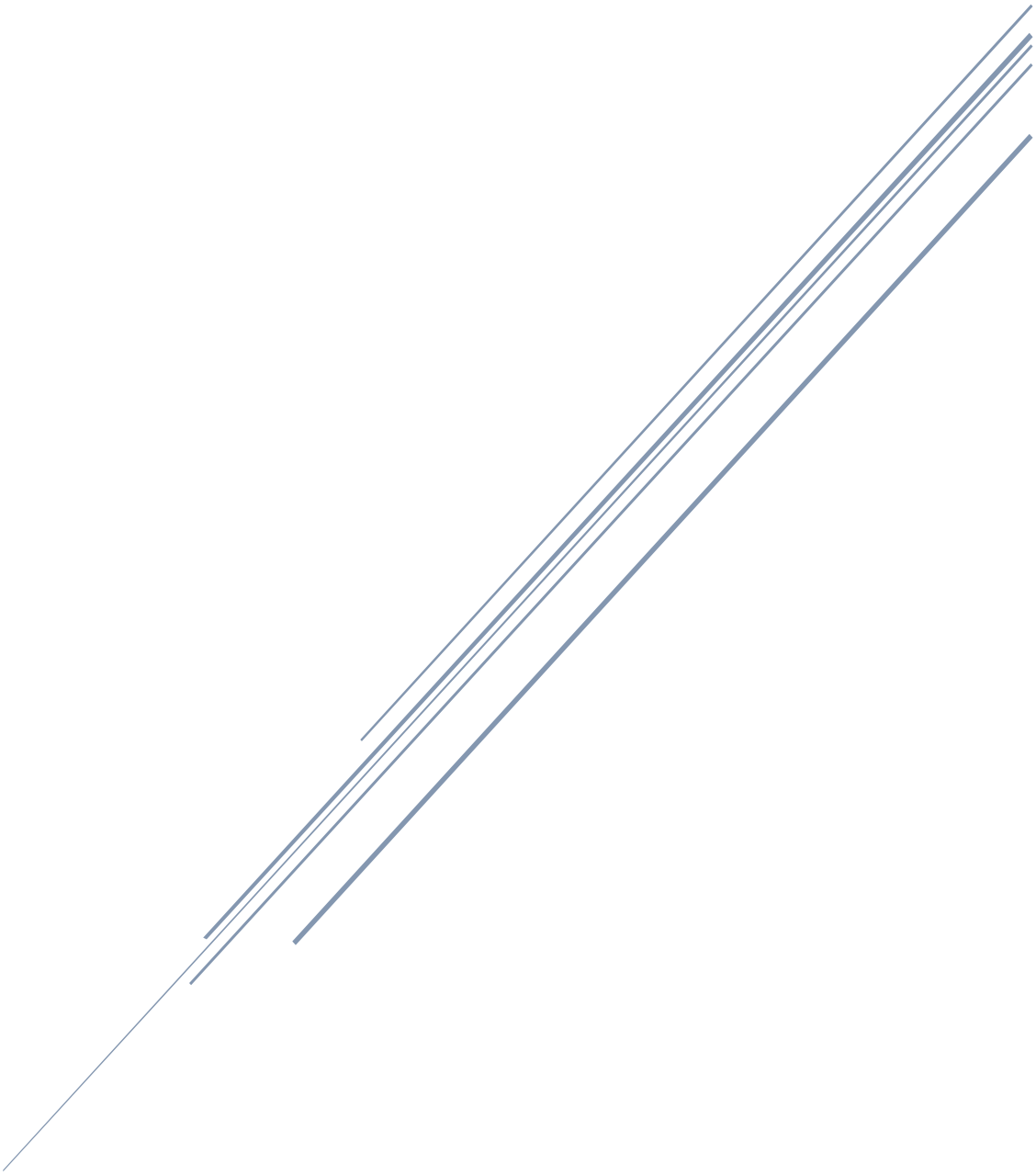
SPACE SHOOTER

Prepared by Viktoria Adamus



Contact Details

Viktoria Adamus H00500182 va4012@hw.ac

Contents

1. Introduction

The aim of this report is to describe the game Space Shooter, a 2D game developed using the C programming language and the Raylib graphical library.

2. Gameplay Description

The game space shooter is lightly inspired by one of the first games, "Space Invaders". The player controls the rotation of a spaceship that is placed in the middle of the game window and shoots blasts from the tip of the nose of the spaceship to eliminate aliens. Aliens are spawning every half-second from the edges of the window and moving towards the spaceship. Score increases by one for every eliminated alien, and when the alien is eliminated, an image of an explosion appears in its place for two seconds. The player has 3 lives and loses one each time an alien reaches the spaceship. When the player loses all lives, the game is over. The play again button can be used to play the game again. The gameplay logic is presented in the following flowchart. The graphics were created using the piskelapp.com website.

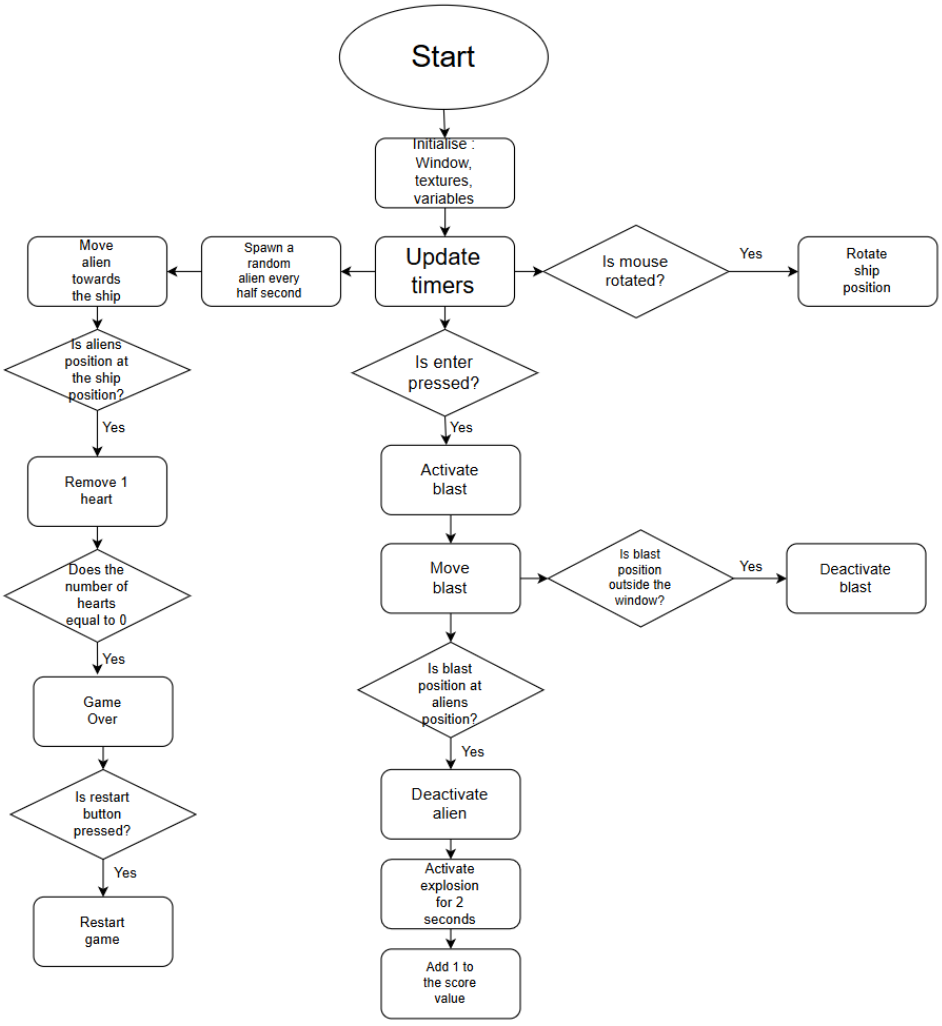


Figure 1. Flowchart 1.

3. Libraries used

This project uses the following libraries to handle graphics, input and mathematical operations:

Raylib

Raylib is used for rendering graphics, handling user input and generating a game window. In this program, the following functions were used for the following applications:

"InitWindow()"- Creating the game window.

"SetTargetFPS()" - Setting frame rate to control game speed.

"WindowShouldClose()" - Closing the window.

"BeginDrawing ()" - Starts drawing graphics.

"EndDrawing()" - ends drawing graphics.

"ClearBackground()" - Fills the background with the selected colour.

"DrawRectangle()" - Draw rectangles for objects like aliens, player, and blasts.

"DrawText()" - Displays text on the screen, used for score value and play again button.

"IsKeyPressed()" - Detects a key pressed event, used for shooting.

"CheckCollisionRecs()" - Checks if two rectangles overlap, used for collisions between aliens and the ship, and aliens and blasts.

Stdlib

Stdlib is a standard library used for general utilities. In this program, the following functions were used for the following applications:

"srand()" – Seeds the random number generator to produce different sequences of random numbers.

"rand()" – Generates a pseudo-random integer, used for spawning aliens.

Math.h

Math.h provides functions used for advanced mathematical calculations. In this program, the following functions were used for the following applications:

"sinf() " and "cosf()" - Calculate the sine and cosine of an angle in radians. Used for the shooting algorithms to shoot the blasts in the right directions.

"atan2f()" -Calculate the angle in radians from a vector x,y. Used for ship rotation.

"sqrtf()" – Calculate the square root of floating-point numbers. Used for aliens moving towards the spaceship after spawning.

Time.h

Time.h is a standard C library to used to handle time and date functions. In this program, the following functions were used for the following applications:

"time()" – provides the current time in seconds. Used to seed the random number generator.

4. User-Defined functions

This section describes the functions created specifically for the space shooter game. For this project, the following functions were created:

InitBlast - initialise all the blast arrays at the start of the game. Sets all blasts inactive at the start of the game. Called in the main loop before the game starts.

InitAliens - initialise all alien-related arrays. Called in the main loop before the game starts.

InitExplosion - initialise explosion effects for aliens. Called in the main loop before the game starts.

GetShipCollision - creates and returns a rectangle centred at the ship position, used for collision detection with aliens. Called for the collision mechanics each frame.

GetBlastRectangle - creates and returns a rectangle at the blast position, used for collision detection with aliens. Called for the collision mechanics each frame.

GetAlienRectangle - creates and returns a rectangle at the alien's position, used for collision detection for both blasts and the ship. Called for the collision mechanics each frame.

GetShipRectangle – creates and returns a rectangle for spaceship drawing. The spaceship is drawn with DrawTexturePro() to enable rotation, this function requires rectangles.

GetShipOrigin - returns the centre point of the ship to ensure the ship rotates around its centre. Called each frame when drawing the ship.

GetAlienType - returns the correct alien texture based on its type, as 3 types of alien textures were used for this game. Called for drawing an alien texture.

5. Game mechanics

Ship rotation

The ship rotation mechanism allows the ship to rotate in the direction of the mouse cursor. The ship rotation is calculated by determining the angle between the ship's position and the mouse cursor and converting it to degrees. The angle is calculated as follows:

$\Theta = tan^{-1}(\frac{y\ mouse\ position - y\ ship\ position}{x\ mouse\ position - x\ ship\ position}) \times \frac{180}{\pi}$

Shooting

The shooting mechanic allows the player to fire blasts in the direction the spaceship is facing. When the enter key is pressed, the program searches for an inactive blast from a fixed-size array. Once found, the blast is positioned at the front of the spaceship. The position of the blast is calculated by adding the rotation of the ship times the nose offset to the fixed x and y positions of the ship. The position of the blast is calculated by the following calculation:

Position of the blast:

$\Theta rad = \Theta deg \times \frac{\pi}{180}$

$Nose\ offset = \frac{height\ of\ the\ ship}{2}$

X position = x position of the ship + sin(Θrad) x Nose offset - $\frac{width\ of\ the\ blast}{2}$

Y position = y position of the ship – cos(Θrad) x Nose offset

Direction of the blast:

X direction = sin(Θrad)

Y direction = - cos(Θrad)

Blasts movement

Once the blast is positioned at the nose of the spaceship, it travels in the direction that the ship is facing at a speed of 10 pixels per frame. When the blast moves outside the window boundaries, it deactivates. The position of the blast is updated each frame by the following calculation:

X position = x direction x speed of the blast (10 pixels per frame)

y position = y direction x speed of the blast (10 pixels per frame)

Time update for spawning aliens

Spawn timer uses GetFrameTime() to store elapsed time each frame as a float. This float is later used to spawn aliens every half-second.

Alien spawning mechanism

A random alien spawns at a random position along the edges of the screen every half-second. Once the Spawn timer is equal to or greater than half a second, the Spawn timer is restarted. After that, the program selects a random edge to spawn an alien using rand() % 4. A switch statement then assigns the alien's x and y position depending on the chosen edge, using a second rand() to pick a random position along that edge. The alien is placed just outside the screen to enter it naturally. After spawning, the program calculates the alien direction that points towards the ship position. The alien direction is calculated as follows:

Distance in y position dy = y ship position – y alien position

Distance in x position dx = x ship position – x alien position

Distance = $\sqrt{dy^2 + dx^2}$

Y direction = $\frac{dy}{distance}$

x direction = $\frac{dx}{distance}$

Alien movement

Once alien is positioned at a random edge it will travel towards the spaceship at speed of 2 pixels per frame. Alien movement is calculated in a way that every alien will hit the ship if it will not get eliminated. Alien position is calculated as follows:

X position = x direction x speed of the alien (2 pixels per frame)

Y position = y direction x speed of the alien (2 pixels per frame)

Collision mechanism

Collision detection is implemented using rectangular hitboxes for aliens, blasts and the ship. The program uses the CheckCollisionRecs() function to determine if two rectangles overlap. When a collision between an alien and a ship occurs, the player loses one live and if the player loses all three lives, the game is over. When a blast and an alien overlap, the alien is eliminated and replaced by a graphic of an explosion for 2 seconds, the blast is deactivated, and a value of 1 is added to the score.

Play again mechanism

When player lives reach zero, the Game Over is set to true, which stops normal gameplay mechanics such as alien spawning and blasts shooting and displays game over text over the screen. The program displays a play again button drawn using a rectangle and text. Activation of the play again button is detected by CheckCollisionPointRecs(), checking overlap between the rectangle and the mouse left click. When the button is pressed, the game is restarted, variables such as score, lives, and spawn timers are restarted, and all active aliens and blasts are cleaned.

6. Design decisions

Rectangles

Rectangles were chosen to represent the ship, aliens and blasts as they are the easiest shape to use in Raylib. They were used in collision mechanics and drawing with functions like CheckCollisionRecs() and DrawTexturePro(). For this simple 2D game, rectangles worked efficiently and were simple to use.

User-defined functions

User-defined functions were used to organise repetitive tasks, to improve code readability, and to make code easier to maintain and update. For instance, functions like GetShipCollision() or GetBlastRectangle() encapsulate specific tasks, so the main loop remains clean and focused on the game logic rather than low-level details.

Arrays

Arrays were used in the game to efficiently manage multiple instances of the same object, such as blasts and aliens. Without arrays, every object would require its own set of variables, which would make the code complicated and hard to handle.

7. Conclusion

The space shooting project demonstrates how a simple 2D game can be created using Raylib with the C programming language. Overall, this project shows how programming concepts like loops, functions and arrays can be combined to create an interactive and functional game.